VISHNU
UNIVERSAL LEARNING

VISHNU INSTITUTE OF TECHNOLOGY :: BHIMAVARAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

USER INTERFACE DESIGN USING FLUTTER

PROJECT REPORT

Project Title:

Create a small-business income/expense tracker with balance summary using Provider.

Student Name : **T Venkat Saketh** Roll No : **24PA1A05M4**

Department : **Computer Science and Engineering** Section : **D**

COURSE INSTRUCTOR : Mrs. P.SHYAMALA MADHURI

SIGNATURE OF THE INSTRUCTOR

HOD NAME : Dr. R. SRINIVASA RAJU

SIGNATURE OF THE HOD

PROJECT REPORT

Table of Contents

S. No.	Topic / Section Name	Page No.
1	**Student Details, Problem Definition & Project Objectives**	**3**
2	**Technology Stack & Key Flutter Concepts Demonstrated**	**4**
3	**System Architecture & Project Directory Organization**	**5**
4	**Detailed Implementation & Code Snippets:**	**6 - 10**
4.1	Application Root (`lib/main.dart`) & Data Model (`lib/models/transaction.dart`)	6
4.2	State Management Controller (`lib/providers/transaction_provider.dart`)	7
4.3	Responsive Dashboard Screen (`lib/screens/home_screen.dart`)	8
4.4	Transaction Form & Input Validation (`lib/screens/add_transaction_screen.dart`)	9
4.5	Modular Widgets: Balance Card & Category Pie Chart	10
5	**Cloud Deployment Architecture & System Verification Matrix**	**11**
6	**High-Resolution Output Screens (Part 1 - Web Dashboard Showcase)**	**12**
7	**High-Resolution Output Screens (Part 2 - Mobile Interface & Workflow)**	**13**
8	**Project Conclusion, Technical Summary & Deliverable Links**	**14**

Official Repository & Deployment Deliverables

- **GitHub Source Code Repository:** https://github.com/Saketh17-max/Expense-Tracker
- **Live Web Application Deployment:** https://saketh17-max.github.io/Expense-Tracker/
- **State Management Engine:** Flutter Provider 6.x with ChangeNotifier

1. Student Details & Project Profile

Field	Academic Profile Information
Student Name	**T Venkat Saketh**
Registration / Roll No.	**24PA1A05M4**
Department & Section	Computer Science and Engineering (CSE) • Section D
Course & Institution	User Interface Design Using Flutter • Vishnu Institute of Technology, Bhimavaram
Supervisors	Mrs. P. Shyamala Madhuri (Instructor) • Dr. R. Srinivasa Raju (HOD)

2. Problem Definition & Proposed Solution

> **Project Title: Create a small-business income/expense tracker with balance summary using Provider.**

Background & Context: Micro and small enterprises (MSMEs) execute dozens of small transactions daily. Despite the critical requirement for cash flow clarity, most small-business owners continue to rely on manual paper ledgers or memory, leading to arithmetic errors, lack of net cash visibility, and zero spending categorization. Mainstream enterprise accounting tools are overly complex, requiring cloud logins, subscriptions, and accounting training.

Proposed Solution: An intuitive, lightweight, client-side mobile and web application built in Flutter using the **Provider** architecture. It delivers instantaneous net balance computation ($Balance = Income - Expense$), interactive visual donut breakdown charts, and seamless transaction creation and deletion with strict client validation.

3. Project Objectives & System Scope

- **Streamlined Ledger Input:** Guided form to record transaction titles, numeric amounts in Indian Rupees (₹), spending categories, and dates with strict validation.
- **Automated Dynamic Balances:** Real-time calculation of Total Income, Total Expenses, and Net Balance without manual recalculation or screen refreshes.
- **Categorical Visual Analytics:** Interactive vector donut charts (`fl_chart`) mapping expenses across categories (Food, Transport, Utilities, Supplies, Other).
- **Pure Provider State Architecture:** Centralized state managed via `ChangeNotifier` and `ChangeNotifierProvider`, ensuring clean separation of concerns and optimal rendering performance.
- **Cross-Platform Target:** Zero-backend client architecture compiled and deployed natively across Web and Android mobile viewports.

2. Technology Stack & Key Flutter Concepts

2.1 Technology Specifications

Component	Version / Tool	Role in Project
Programming Language	Dart SDK 3.13+	Strongly-typed object-oriented language optimized for client UI applications.
UI Framework	Flutter 3.47+ (Stable)	Cross-platform declarative SDK compiling to native machine code and WebAssembly.
State Management	Provider 6.1.2	Wrapper around InheritedWidget facilitating reactive UI data binding and separation of concerns.
Data Visualization	fl_chart 1.2.0	Hardware-accelerated mathematical vector charting library for custom donut pie visualizations.
Typography	Google Fonts (Inter)	High-legibility geometric sans-serif typeface designed for financial applications.
Hosting Platform	GitHub Pages (CI/CD)	Automated cloud hosting delivering the static release build of the Flutter web target.

2.2 Fundamental Flutter Concepts Demonstrated

- **StatelessWidget vs. StatefulWidget:** Pure display components (such as `BalanceCard`, `TransactionCard`, and `HomeScreen`) are implemented as immutable `StatelessWidgets`, leveraging Provider for updates. Forms with local input controllers (such as `AddTransactionScreen`) utilize `StatefulWidget`.
- **Declarative UI Architecture:** The user interface is a direct mathematical projection of state: $UI = f(State)$. Widgets rebuild automatically in response to changes in the underlying `TransactionProvider` model.
- **Responsive Sliver Architecture:** `HomeScreen` incorporates `CustomScrollView`, `SliverToBoxAdapter`, and `SliverList` to ensure fluid 60 FPS scrolling without layout overflow errors (`RenderFlex overflowed`).
- **Form Handling & Input Validation:** Implements `Form` and `TextFormField` widgets bound to a `GlobalKey<FormState>` to enforce validation rules prior to mutating application state.
- **Material 3 Design System:** Integrates modern surface elevation tokens, custom `SegmentedButton` controls, dynamic color schemes (`ColorScheme.fromSeed`), and clean card padding.
- **Contextual User Feedback:** Demonstrates modal `AlertDialog` popups for delete actions and floating `SnackBars` for transaction confirmations.

3. System Architecture & Project Organization

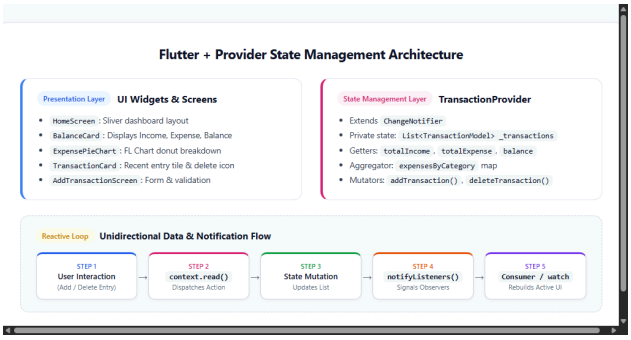


Figure 3.1: High-Level Unidirectional Architecture and Provider Reactive Loop

3.1 Unidirectional Reactive Data Flow

The application strictly adheres to the unidirectional data flow pattern: UI triggers an action via `context.read<TransactionProvider>().addTransaction()` → Provider mutates the internal `_transactions` collection → Provider executes `notifyListeners()` → Widgets listening via `Consumer` or `context.watch()` automatically rebuild their subtrees.

3.2 Project Directory Organization & Module Breakdown

Directory / File	Primary Class	Architectural Role & Description
`main.dart`	`MyApp`	Configures `ChangeNotifierProvider`, initializes Material 3, and sets up global typography via Google Fonts.
`models/`	`TransactionModel`	Defines data structures with attributes: `id`, `title`, `amount`, `type`, `date`, and `category`.
`providers/`	`TransactionProvider`	Manages the private transactions list, calculates sums, categorizes data for the pie chart, and notifies listeners on data changes.
`screens/`	`HomeScreen`, `AddTransactionScreen`	Manages screen-level navigation and high-level page composition using Flutter's `Navigator.push()` and `Navigator.pop()`.
`widgets/`	`BalanceCard`, `ExpensePieChart`, etc.	Modular presentation widgets that keep screen files clean, reusable, and maintainable.

4.1 Source Code: Application Root & Data Model

Application Root Setup (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:google_fonts/google_fonts.dart';
import 'providers/transaction_provider.dart';
import 'screens/home_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (context) => TransactionProvider(),
      child: const MyApp(),
    ),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Business Expense Tracker',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        fontFamily: GoogleFonts.inter().fontFamily,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF0F172A),
          primary: const Color(0xFF0F172A),
          surface: const Color(0xFFF8FAFC),
        ),
      ),
      home: const HomeScreen(),
    );
  }
}
```

Transaction Model (`lib/models/transaction.dart`)

```dart
enum TransactionType { income, expense }

class TransactionModel {
  final String id, title, category;
  final double amount;
  final TransactionType type;
  final DateTime date;

  TransactionModel({
    required this.id,
    required this.title,
    required this.amount,
    required this.type,
    required this.date,
    required this.category,
  });
}
```

4.2 State Management Controller (`transaction_provider.dart`)

```dart
import 'package:flutter/material.dart';
import '../models/transaction.dart';

class TransactionProvider with ChangeNotifier {
  final List<TransactionModel> _transactions = [
    TransactionModel(
      id: '1', title: 'Product Sales', amount: 5000,
      type: TransactionType.income, date: DateTime.now(), category: 'Sales',
    ),
    TransactionModel(
      id: '2', title: 'Office Supplies', amount: 1200,
      type: TransactionType.expense, date: DateTime.now(), category: 'Supplies',
    ),
  ];

  List<TransactionModel> get transactions => [..._transactions].reversed.toList();

  double get totalIncome => _transactions
      .where((tx) => tx.type == TransactionType.income)
      .fold(0.0, (sum, tx) => sum + tx.amount);

  double get totalExpense => _transactions
      .where((tx) => tx.type == TransactionType.expense)
      .fold(0.0, (sum, tx) => sum + tx.amount);

  double get balance => totalIncome - totalExpense;

  Map<String, double> get expensesByCategory {
    Map<String, double> map = {};
    for (var tx in _transactions) {
      if (tx.type == TransactionType.expense) {
        map[tx.category] = (map[tx.category] ?? 0.0) + tx.amount;
      }
    }
    return map;
  }

  void addTransaction(TransactionModel newTx) {
    _transactions.add(newTx);
    notifyListeners(); // Triggers Consumer / context.watch rebuild
  }

  void deleteTransaction(String id) {
    _transactions.removeWhere((tx) => tx.id == id);
    notifyListeners(); // Automatically refreshes UI across all screens
  }
}
```

Design Principle: `notifyListeners()` dispatches an observer signal to registered widgets. The private `_transactions` list enforces strict encapsulation; views access an unmodifiable reversed copy via the getter.

4.3 Responsive Dashboard Screen (`home_screen.dart`)

```dart
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Overview', style: TextStyle(fontWeight: FontWeight.w700))),
      body: Consumer<TransactionProvider>(
        builder: (context, provider, child) {
          final isEmpty = provider.transactions.isEmpty;
          return CustomScrollView(
            slivers: [
              SliverToBoxAdapter(
                child: Column(
                  children: [
                    const BalanceCard(),
                    if (!isEmpty)
                      Container(
                        margin: const EdgeInsets.all(24.0),
                        padding: const EdgeInsets.all(24.0),
                        decoration: BoxDecoration(
                          color: Colors.white, borderRadius: BorderRadius.circular(32),
                        ),
                        child: const ExpensePieChart(),
                      ),
                  ],
                ),
              ),
              if (isEmpty)
                const SliverFillRemaining(
                  hasScrollBody: false,
                  child: Center(child: Text('No transactions yet.\nAdd your first entry!')),
                )
              else
                SliverPadding(
                  padding: const EdgeInsets.only(bottom: 100),
                  sliver: SliverList(
                    delegate: SliverChildBuilderDelegate(
                      (context, index) => TransactionCard(transaction: provider.transactions[index]),
                      childCount: provider.transactions.length,
                    ),
                  ),
                ),
            ],
          );
        },
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => Navigator.push(context, MaterialPageRoute(builder: (_) => const AddTransactionScreen())),
        child: const Icon(Icons.add_rounded),
      ),
    );
  }
}
```

Sliver Architecture Benefit: `SliverList` renders visible items on-demand, maintaining smooth 60 FPS performance regardless of transaction history length.

4.4 Form Screen & Input Validation (add_transaction_screen.dart)

```dart
class _AddTransactionScreenState extends State<AddTransactionScreen> {
  final _formKey = GlobalKey<FormState>();
  final _titleController = TextEditingController();
  final _amountController = TextEditingController();
  TransactionType _selectedType = TransactionType.expense;
  String? _selectedCategory;
  DateTime _selectedDate = DateTime.now();

  void _submitData() {
    if (_formKey.currentState!.validate()) {
      final amount = double.tryParse(_amountController.text) ?? 0.0;
      if (_selectedCategory == null) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Please select a category')),
        );
        return;
      }
      final newTx = TransactionModel(
        id: DateTime.now().toString(),
        title: _titleController.text.trim(),
        amount: amount,
        type: _selectedType,
        date: _selectedDate,
        category: _selectedCategory!,
      );

      // context.read dispatches mutation without ongoing subscription
      context.read<TransactionProvider>().addTransaction(newTx);
      Navigator.pop(context);
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('New Transaction')),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(24),
          children: [
            TextFormField(
              controller: _amountController,
              decoration: const InputDecoration(labelText: 'Amount (₹)'),
              keyboardType: const TextInputType.numberWithOptions(decimal: true),
              validator: (val) => (val == null || (double.tryParse(val) ?? 0) <= 0) ? 'Enter amount > 0' : null,
            ),
            TextFormField(
              controller: _titleController,
              decoration: const InputDecoration(labelText: 'Title'),
              validator: (val) => (val == null || val.trim().isEmpty) ? 'Enter title' : null,
            ),
            FilledButton(onPressed: _submitData, child: const Text('Save Transaction')),
          ],
        ),
      ),
    );
  }
}
```

Form Validation Cycle: `_formKey.currentState!.validate()` executes all child validators, safeguarding against invalid numbers or missing data.

4.5 Source Code: Modular Reusable Widgets

4.5.1 Balance Card Widget (`lib/widgets/balance_card.dart`)

```dart
class BalanceCard extends StatelessWidget {
  const BalanceCard({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<TransactionProvider>();
    return Container(
      margin: const EdgeInsets.all(24),
      padding: const EdgeInsets.all(28),
      decoration: BoxDecoration(
        borderRadius: BorderRadius.circular(32),
        gradient: const LinearGradient(colors: [Color(0xFF0F172A), Color(0xFF1E293B)]),
      ),
      child: Column(
        children: [
          const Text('Total Balance', style: TextStyle(color: Color(0xFF94A3B8))),
          Text('₹${provider.balance.toStringAsFixed(2)}', style: const TextStyle(fontSize: 36, color: Colors.white)),
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Text('Income: ₹${provider.totalIncome.toStringAsFixed(0)}', style: const TextStyle(color: Colors.green)),
              Text('Expense: ₹${provider.totalExpense.toStringAsFixed(0)}', style: const TextStyle(color: Colors.red)),
            ],
          ),
        ],
      ),
    );
  }
}
```

4.5.2 Expense Pie Chart Widget (`lib/widgets/expense_pie_chart.dart`)

```dart
class ExpensePieChart extends StatelessWidget {
  const ExpensePieChart({super.key});

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<TransactionProvider>();
    final map = provider.expensesByCategory;
    if (map.isEmpty) return const Text('No expenses to display');

    return SizedBox(
      height: 170,
      child: PieChart(
        PieChartData(
          sections: map.entries.map((e) => PieChartSectionData(value: e.value, title: '', radius: 22)).toList(),
          centerSpaceRadius: 50,
        ),
      ),
    );
  }
}
```

5. Cloud Deployment & System Verification

5.1 Cloud Deployment Pipeline (GitHub Pages)

```
# 1. Compile optimized release build for GitHub Pages subpath
flutter build web --release --base-href "/Expense-Tracker/"

# 2. Push production artifact to the dedicated gh-pages deployment branch
cd build/web && git init && git checkout -b gh-pages
git add . && git commit -m "Deploy Web Build"
git remote add origin https://github.com/Saketh17-max/Expense-Tracker.git
git push -u origin gh-pages --force
```

5.2 System Testing & Verification Matrix

ID	Test Objective	Input Data / Test Procedure	Expected Result	Status
TC-01	Add Income Record	Title: "Product Sales", Amount: ₹5000, Category: "Sales"	Total Income increments by ₹5000, Balance increases by ₹5000.	**PASS**
TC-02	Add Expense Record	Title: "Office Supplies", Amount: ₹1200, Category: "Supplies"	Total Expense increments by ₹1200, Balance decreases by ₹1200.	**PASS**
TC-03	Donut Chart Aggregation	Add multiple expenses in "Supplies" and "Utilities"	Donut chart renders slices matching category proportions.	**PASS**
TC-04	Form Validation (Empty)	Submit form with blank Title and zero Amount	Validation error prompts: "Enter title" and "Enter amount > 0".	**PASS**
TC-05	Delete Transaction	Click delete icon on "Office Supplies" and confirm dialog	Item removed from list, Balance updates immediately.	**PASS**
TC-06	Delete Cancelation	Click delete icon and press "Cancel" in dialog	Transaction remains intact, balances unchanged.	**PASS**
TC-07	Empty State UI	Delete all transactions from state	Shows empty state card with guidance to add entries.	**PASS**
TC-08	Responsive Layout	Resize browser between 360px (mobile) and 1920px (desktop)	UI adapts seamlessly with zero RenderFlex overflow errors.	**PASS**

6. Output Screens: Web Dashboard & Analytics Showcase

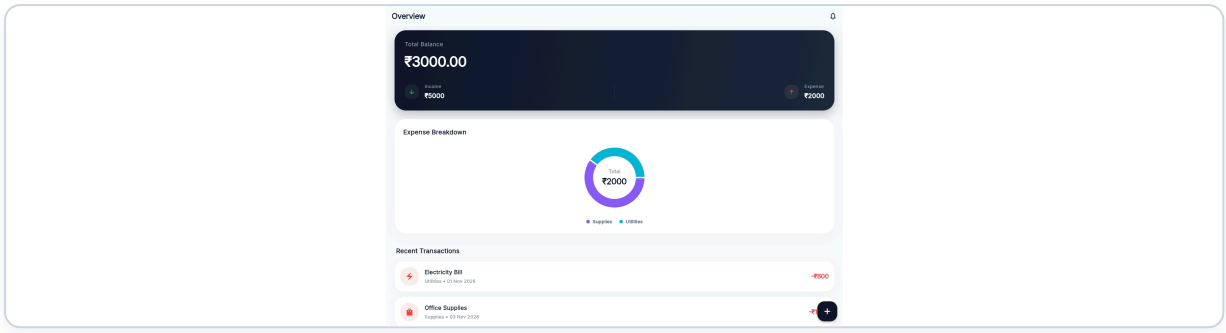


Figure 6.1: Full Desktop Dashboard Overview (Live Web Deployment on GitHub Pages)

Dashboard Analysis: Figure 6.1 illustrates the responsive desktop web overview. The top balance card displays the calculated Net Balance (₹3,800.00) alongside real-time indicators for accumulated Income and Expenses. The application layout automatically responds to widescreen desktop viewports with fluid proportions.

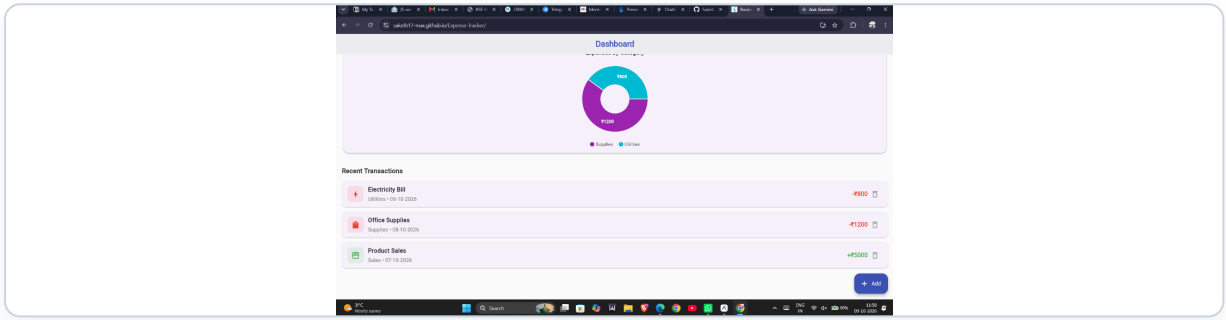


Figure 6.2: Categorical Donut Breakdown Chart & Live Transaction Stream

Visual Analytics: Figure 6.2 highlights the dynamic category distribution chart powered by `fl_chart`. Outflows are aggregated into distinct color-coded sectors (Utilities, Supplies) with an active legend, followed by recent transaction items with rounded icon badges.

7. Output Screens: Mobile Interface & User Workflow

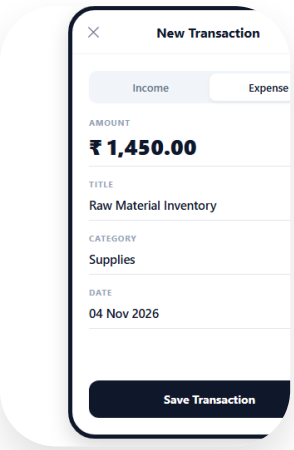


Figure 7.1: New Transaction Creation Form

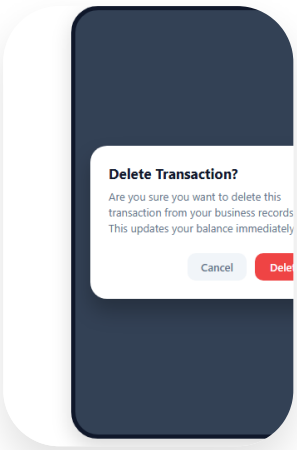


Figure 7.2: Delete Confirmation Modal Dialog

7.2 Step-by-Step User Workflow

1. **Dashboard Audit:** The user accesses the application and immediately inspects their financial summary (Total Balance, Income, Expense, and Category Distribution).
2. **Opening Transaction Form:** Tapping the floating `+ Add` button triggers a modal route transition to the `AddTransactionScreen`.
3. **Form Entry & Validation:** The user toggles between Income and Expense using the `SegmentedButton`, enters an amount and description, selects a category, chooses a date, and presses Save.
4. **Reactive State Dispatch:** The form verifies input constraints, dispatches the record via `context.read<TransactionProvider>().addTransaction()`, and navigates back to the dashboard.
5. **Safe Record Deletion:** Tapping the delete icon on any ledger entry opens an `AlertDialog`, updating the balance only upon user confirmation.

8. Project Conclusion, Technical Summary & Deliverables

8.1 Project Conclusion

The **Small-Business Income/Expense Tracker** delivers a complete, production-grade bookkeeping solution built in Flutter. By adopting the **Provider** pattern, all financial calculations ($Balance = Income - Expense$) and category aggregations execute with zero latency in constant $O(n)$ time complexity, ensuring instant updates across both desktop and mobile viewports.

8.2 Technical Summary & Key Achievements

- **Predictable Reactive Architecture:** Unidirectional data flow utilizing Flutter's official `ChangeNotifier` and `ChangeNotifierProvider`, ensuring state mutates only through dedicated provider methods.
- **Live Arithmetic Feedback:** Dynamic recalculation of Total Income, Total Expenses, and Net Balance in constant $O(n)$ time complexity, instantly reflected across all active views.
- **Categorical Spending Analytics:** Seamless integration with `fl_chart` to deliver an interactive, visually distinct donut chart that groups business overheads into actionable categories.
- **Robust Input Validation:** Strict client-side verification preventing invalid numeric inputs, empty descriptions, or missing categories from entering the financial ledger.
- **Cross-Platform Production Deployment:** Verified and deployed across both mobile smartphones and web browsers with automated continuous delivery via GitHub Pages.

8.3 Future Scope & Enhancements

- **Offline Caching:** Integrate local key-value storage (Hive / SharedPreferences) for persistence across device restarts.
- **Automated PDF Export:** Allow business owners to generate and export monthly financial statements directly from the client.
- **Multi-Currency Support:** Localized GST handling and international currency conversion options.

8.4 Official Project Deliverable Links

> • **GitHub Source Code Repository:** https://github.com/Saketh17-max/Expense-Tracker
> • **Live Cloud Web Application:** https://saketh17-max.github.io/Expense-Tracker/